

Estado del Arte: AUTOMATIZACION ROBOTICA DE PROCESOS (RPA)

Autor: Jorge Abraham Ayala Mendoza

Fecha: 12 Junio 2026

I. Resumen

La Automatización Robótica de Procesos (RPA) ha dejado de ser una simple capa de emulación de periféricos para transformarse en un componente crítico de las arquitecturas de software empresarial distribuidas. Este documento presenta una revisión crítica del estado del arte de RPA, enfocándose en la transición técnica desde los scripts basados en el reconocimiento de selectores de interfaz de usuario hacia ecosistemas guiados por eventos (Event-Driven Automation) e integraciones híbridas de bajo acoplamiento mediante servicios web y APIs. Se analizan los patrones de diseño comunes empleados para mitigar la fragilidad de los sistemas automatizados, se evalúa una nueva selección de literatura académica contemporánea que aborda el impacto de la inteligencia artificial descentralizada en flujos no estructurados, y se clasifica el panorama tecnológico actual dividiendo las soluciones comerciales de las plataformas de orquestación basadas puramente en código fuente.

Palabras clave: *Automatización asíncrona, Event-driven RPA, Orquestación de flujos, Robustez de software, Arquitectura de software, APIs REST.*

II. Introducción

En la práctica de la ingeniería de software actual, uno de los mayores problemas operativos es la persistencia de sistemas heredados (legacy) que carecen de interfaces programables estándar. Históricamente, la transferencia de información entre estos sistemas requería la intervención de un operador que ejecutara secuencias mecánicas repetitivas. RPA surgió como una técnica pragmática para interceptar estas tareas simulando las acciones humanas directamente en el plano de la interfaz de usuario.

Sin embargo, desde la perspectiva de un desarrollador en formación, los primeros experimentos prácticos revelan una dura realidad: el software diseñado para interactuar visualmente con otros softwares es inherentemente inestable. Cualquier alteración menor en el

árbol de nodos de una aplicación web, un retraso en la carga de red o un cambio menor en el renderizado destruye la lógica del bot. Por lo tanto, el estado del arte de esta disciplina no se centra hoy en cómo clonar clics de forma más rápida, sino en cómo diseñar flujos de automatización concurrentes, tolerantes a fallos y desacoplados del canal visual. El propósito de este trabajo es documentar la evolución metodológica del área, contrastando los enfoques tradicionales frente a las arquitecturas reactivas modernas guiadas por código y eventos.

III. Marco Conceptual y Arquitectónico

3.1. Taxonomía de Control en Ecosistemas Automatizados

Para modelar el comportamiento de un bot dentro de un ciclo de software, es imperativo fragmentar su flujo en componentes de control bien definidos. La siguiente tabla describe cómo interactúan estas estructuras lógicas en una implementación contemporánea:

Módulo Arquitectónico	Función Técnica	Implementación del Lado del Desarrollador
Orquestador / Motor Central	Administra el ciclo de vida del proceso, balancea las colas de mensajes y registra la telemetría del bot.	Monitoreo centralizado de logs de error y tasas de éxito en tiempo real.
Listener / Disparador Reactivo	Componente asíncrono que permanece a la escucha de mutaciones de estado o señales externas.	Un webhook que captura un JSON entrante desde un sistema de pasarela de pagos.
Manejador de Excepciones	Lógica dedicada a interceptar caídas lógicas y aplicar políticas de reintento (retry-policies) automáticas.	Bloques Try-Catch estructurados con estrategias de retroceso exponencial (exponential backoff).
Capa de Persistencia	Almacén temporal o definitivo que asegura el estado de la transacción ante una interrupción abrupta del flujo.	Escritura de estados parciales en una base de datos relacional ligera o caché en memoria.

3.2. Ruptura de Paradigma: UI-Driven vs. API-Driven Automation

El análisis técnico obliga a trazar una línea divisoria entre el RPA primitivo y las tendencias de diseño actuales:

- **Enfoque Orientado a UI (UI-Driven):** El bot está acoplado al Front-End. Emplea técnicas como análisis de selectores DOM, reconocimiento óptico de caracteres (OCR) y coordenadas relativas. Es la única vía viable ante aplicaciones de caja negra o entornos de escritorio remoto de tipo Citrix, pero su costo de mantenimiento en producción es críticamente alto.
- **Enfoque Orientado a API (API-Driven):** El bot se comunica directamente con las capas de servicios expuestas (Back-End). Realiza peticiones HTTP, manipula payloads

estructurados (JSON/XML) e interactúa de manera directa con colas de mensajería. Este enfoque anula la capa de presentación, reduciendo la latencia de ejecución a milisegundos y garantizando que el bot no falle aunque la interfaz del software cambie por completo.

3.3. Patrones de Diseño Estructurales

En el desarrollo práctico, estructurar el código sin patrones claros da lugar a bots difíciles de depurar. Los tres patrones esenciales aplicados en la automatización moderna son:

1. **Procesamiento en Lotes Asíncronos (Batch Processing):** Minimiza las conexiones concurrentes agrupando las transacciones para ejecutarlas en intervalos de baja carga de servidores.
2. **Arquitectura Basada en Micro-Tareas:** Divide un proceso complejo en pequeños scripts independientes que se comunican mediante un bus de datos comunes. Si un script falla, los demás permanecen estables.
3. **Intervención Humana bajo Excepción (Human-in-the-loop):** El robot procesa todas las tareas estándar de forma desatendida. Cuando el bot se topa con un caso ambiguo o datos corruptos que no cumplen con las reglas lógicas, aísla la transacción, genera un ticket de alerta y continúa procesando el resto de la cola sin detenerse.

IV. Trabajos Relacionados y Análisis de Literatura Diferenciada

La investigación científica actual aborda el fenómeno de la automatización desde perspectivas metodológicas muy distintas a las de los manuales de uso comerciales:

Leno et al. [1] proponen un enfoque avanzado denominado "Robotic Process Mining" (Minería de Procesos Robóticos). Su investigación se enfoca en cómo utilizar algoritmos de aprendizaje para analizar los logs de interacción de los empleados humanos e identificar de forma automática cuáles subprocesos son candidatos óptimos para ser traducidos a código de automatización, mitigando el sesgo humano en el diseño inicial del flujo.

Anagnoste [2] examina los costos ocultos de mantenimiento en las etapas post-implementación de RPA. El autor demuestra empíricamente que las organizaciones subestiman la velocidad con la que las aplicaciones de terceros se actualizan, concluyendo que las arquitecturas que carecen de un diseño modular robusto terminan costando más en horas de desarrollo y soporte técnico de lo que ahorran operativamente en su primer año de ejecución.

Por otro lado, **Geyer-Klingenberg et al. [4]** analizan la intersección crítica entre RPA y la Minería de Datos Corporativos. Señalan que la automatización superficial actúa muchas veces como un "parche temporal" sobre infraestructuras de software mal estructuradas, lo que puede desincentivar a las empresas a realizar migraciones profundas hacia arquitecturas modernas

basadas en microservicios nativos, perpetuando el uso de sistemas obsoletos.

V. Clasificación Teconológica del Ecosistema de Herramientas

El ecosistema actual de desarrollo puede dividirse drásticamente según su nivel de abstracción y el paradigma de desarrollo que implementa:

Categoría Técnica	Plataformas Representativas	Mecanismo de Control y Desarrollo
Entornos Low-Code / No-Code	Microsoft Power Automate, Zapier	Uso de bloques lógicos visuales y conectores abstractos nativos. Ideal para implementaciones veloces a nivel de usuario de negocio, pero restrictivo para inyectar algoritmos complejos o lógica de control personalizada.
Orquestadores Visuales Flexibles	n8n, Node-RED	Flujos visuales basados en grafos dirigidos por nodos donde cada caja lógica expone y procesa código directo (JavaScript / Python). Ofrece un punto medio excelente para gestionar payloads JSON pesados y manipular webhooks de forma ágil.
Frameworks de Código Puro (Code-as-Configuration)	Prefect, Apache Airflow, Robot Framework	Desarrollo nativo en Python o lenguajes específicos. Toda la automatización es tratada como código fuente de software real: permite el uso de pruebas unitarias, control de versiones estricto con Git, despliegue continuo (CI/CD) e inyección de dependencias.

VI. La Frontera Técnica: Hiperautomatización y Agentes Autónomos

La tendencia de desarrollo que está redefiniendo el campo de estudio es la sustitución de las reglas lógicas cableadas (if-else estáticos) por componentes cognitivos asíncronos. En los escenarios tradicionales, si un cliente enviaba una solicitud con datos no estructurados —como una descripción narrativa libre dentro de un cuerpo de texto—, el robot tradicional fallaba de inmediato al no encontrar una coincidencia exacta de caracteres en sus condicionales.

La integración con arquitecturas de Modelos de Lenguaje Grandes (LLMs) permite añadir "nodos de inferencia" en medio de la tubería de ejecución (pipeline). El robot clásico extrae los

archivos planos o el texto libre, lo envía mediante una llamada de API a un modelo entrenado de lenguaje, y este último devuelve un objeto JSON estrictamente tipado y validado que contiene los datos esenciales clasificados por intención. Con esto, el flujo automatizado vuelve a tomar el control estricto para interactuar con las bases de datos locales, abriendo la puerta al desarrollo de agentes autónomos reactivos capaces de operar de punta a punta.

VII. Conclusiones

El análisis sistemático del estado del arte de RPA demuestra de manera contundente que la disciplina ha migrado hacia estándares rigurosos de la ingeniería de software. La noción rudimentaria de construir bots basados únicamente en capturas de pantalla y simulación mecánica de clics ha quedado relegada a soluciones temporales debido a su fragilidad innata frente a los cambios de interfaz.

Para quienes estudiamos computación, la clave del éxito en este campo radica en dominar los patrones de arquitectura desacoplados y la comunicación por backend mediante APIs y eventos masivos. Diseñar automatizaciones bajo este enfoque garantiza soluciones con alta tolerancia a fallos, facilidad para el mantenimiento del código a largo plazo y una escalabilidad real capaz de incorporar las tecnologías cognitivas y de procesamiento inteligente de datos que demanda el mercado moderno.

VIII. Referencias

- [1] V. Leno, S. J. van Zelst, M. La Rosa, and W. M. P. van der Aalst, "Robotic Process Mining: Vision and Research Challenges," *Business & Information Systems Engineering*, vol. 63, no. 3, pp. 201–214, 2021.
- [2] S. Anagnoste, "Robotic Process Automation - The next major revolution in Germany's digital transformation," *Management & Marketing. Challenges for the Knowledge Society*, vol. 13, no. 4, pp. 1122–1135, 2018.
- [3] C. Cabanillas, A. Di Ciccio, M. Resinas, and A. Ruiz-Cortés, "Towards Information Systems for Process-Aware Robotic Process Automation," in *International Conference on Business Process Management*, Springer, 2020, pp. 135–147.
- [4] J. Geyer-Klingenberg, J. Nakpodia, and F. S. van der Heijden, "When Robotic Process Automation meets Process Mining," *Computers in Industry*, vol. 119, p. 103211, 2020.
- [5] A. Asatiani and E. Penttinen, "Turning robotic process automation into intelligent automation: A review of current architectural trends," *Journal of Information Technology Teaching Cases*, vol. 12, no. 1, pp. 45–56, 2022.